

SATIDA SOOKPONG

Tokenization

1323326 การประมวลผลภาษาธรรมชาติ (Natural Language Processing)
1-0303 Computer & Sound; Lab 2

Text as data



Words

"playing"

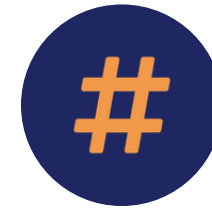
Smallest meaningful standalone units in a sentence.



Morphemes

"play" + "-ing"

Smallest units of meaning inside a word (roots, affixes, suffixes).



Unicode

U+0041 = "A"

Each character mapped to a universal code point — the foundation of digital text.



Subword (BPE)

"un" + "play" + "able"

Splits rare words into frequent subword pieces learned from a corpus.

Text as data

หน่วยพื้นฐานของข้อความ

- Corpus ชุดข้อความ/คลังข้อมูลที่ใช้ฝึกโมเดล
 - Brown Corpus
 - Wall Street Journal
- Token หน่วยย่อยของข้อความ
 - คำ
 - สัญลักษณ์
 - เครื่องหมายวรรคตอน
- Type คำที่แตกต่างกันในคลังข้อมูล (distinct word types)
- Word Form รูปคำที่ปรากฏจริงในคลังข้อมูล (ผันรูป)
- Stem / Lemma รากศัพท์ (compute) กับรูปพจนานุกรม (Lemma) ของคำที่ผันรูป (computers, computation)

Text as data

Normalization

- Raw text:
 - ตัวพิมพ์ใหญ่-เล็ก
 - เครื่องหมายวรรคตอน
 - คำย่อ
 - อีโมจิ
- Text Normalization Process: make it easy to process
- eg: Penn Treebank tokenization — rule-based text preprocessing method

Normalization

Normalization Pipeline

- 1 Lowercasing — Convert all text to lowercase: "Hello" → "hello"
- 2 Punctuation Removal — Strip or separate punctuation from words
- 3 Stop Word Removal — Remove common words: "the", "is", "at"
- 4 Stemming / Lemmatization — "running" → "run"; reduces inflections
- 5 Unicode Normalization — NFC/NFD: unify accented characters to one form

Regular Expressions (Regex)



What is Regex?

online - <https://regex101.com/>

`\w+`

Match a word token

`\d{4}-\d{2}-\d{2}`

Match a date (YYYY-MM-DD)

`[A-Z][a-z]+`

Match a capitalized word (NER hint)

`\s+`

Split on whitespace

Regular Expressions (Regex)

Anatomy of an Expression

[]

Character class — [aeiou] matches any vowel

* + ?

Quantifiers — "a+" means one or more "a"s

^ \$

Anchors — ^Hello matches start of string

()

Groups — (ab)+ captures repeating group

|

Alternation — "cat|dog" matches either word

\w \d \s

Shorthand classes — word, digit, whitespace

Tokenization



Tokenization = การแบ่งข้อความออกเป็นหน่วยย่อย (tokens) เช่น คำ, สัญลักษณ์, หรือ หน่วยย่อยของคำ เพื่อให้คอมพิวเตอร์นำไปประมวลผลต่อได้



“How long is this text?”

- นับหน่วยอย่างไร ขึ้นกับวิธี tokenize
- 1 token $\neq$ 1 word



“don’t” = 1 token หรือ 2?

do + n’t $\rightarrow$ ส่งผลต่อโมเดลที่ตามมา



Fixed tokenization

อัลกอริทึมส่วนใหญ่ (เช่น perplexity) ต้องการการแบ่ง token ที่คงที่

Tokenization Technique



Word-Based Tokenization

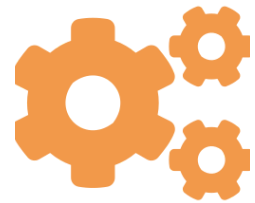
- Split on whitespace or punctuation. "I love NLP" → ["I", "love", "NLP"]
- Simple but fails on contractions, compounds, and **Out of Vocabulary (OOV)** words.
- Example: "don't" → ["don", "", "t"]



Character-Based

- Split on character "I love NLP" → ["I", " ", "l", "o", "v", "e", " ", "N", "L", "P"]
- Meaning lost, Text is too long

Tokenization Technique



Subword Tokenization Methods

BPE: Byte Pair Encoding

iteratively merges frequent character pairs into subwords.

WordPiece: Like BPE but maximises language-model likelihood; used in BERT.

Unigram: Probabilistic model; learns vocab by pruning a large initial set.

Word-based → Subword (BPE / WordPiece)



Word-based

แบ่งตามคำเต็ม

- ปัญหา *OOV* - *vocab* ใหญ่ เกิน



Character-based

แบ่งเป็นตัวอักษร

- ไม่มี *OOV* แต่ลำดับยาวมาก
- เสียความหมายเชิงคำ



Subword (BPE/WordPiece)

แบ่งตามหน่วยย่อย
เรียนรู้จากข้อมูล

- สมดุลระหว่างขนาด *vocab* กับความยาวลำดับ
- มาตรฐานของ *LLM* ปัจจุบัน

BPE Training

Corpus: new (×2), renew (×2), set (×1), reset (×1)

0	เริ่มต้น	n e w / r e n e w / s e t / r e s e t	vocab: e, n, r, s, t, w
1	นับคู่อักษรที่อยู่ติดกันบ่อยที่สุด	(n, e) WU 4 ครั้ง (ใน new×2 และ renew×2) ne w / r e ne w / s e t / r e s e t	vocab: e, n, r, s, t, w, "ne"
2	นับซ้ำหลัง merge	(ne, w) WU 4 ครั้ง new / r e new / s e t / r e s e t	vocab: e, n, r, s, t, w, "ne", "new"
3	ทำซ้ำ	(r, e) WU 3 ครั้ง new / re new / s e t / re s e t	vocab: e, n, r, s, t, w, "ne", "new", "re" → "renew"
4	ทำซ้ำกับ set/reset	(s, e) แล้ว (se, t)	merge → "se" → "set"

BPE Encoder และการใช้งานจริง

การเข้ารหัส (Encoding)

- แบ่งประโยคทดสอบเป็นตัวอักษรก่อน
- ใช้กฎการ merge ที่เรียนรู้จาก training ไล่ทีละกฎ ตามลำดับที่เรียนรู้มา
- ความถี่ใน test data ไม่มีผล — ใช้แค่ลำดับกฎจาก training เท่านั้น
- ทุกคำที่ไม่เคยเห็น (unseen) ยังถูกแทนด้วยลำดับ subword ที่รู้จักได้เสมอ

ในทางปฏิบัติ

- รับบน UTF-8 bytes (byte-level BPE) เพื่อรองรับทุกภาษา/อักขระ รวมถึงภาษาไทย
- จำนวน merge ทั่วไป: 50,000 / 100,000 / 200,000 tokens (เช่น GPT-4o ~200K)
- ไม่มี unknown token อีกต่อไป — เพราะมีค่า byte พื้นฐานแค่ 256 ค่า
- ระบบหลายภาษาอาจมี token ถูกครอบงำโดยภาษาอังกฤษเป็นหลัก ทำให้ภาษาที่ resource น้อยเสียเปรียบ

BPE vs. WordPiece vs. Unigram LM (ULM)

วิธี	หลักการเลือกคู่ที่จะ merge	ใช้งานใน
BPE	เลือกคู่ token ที่ปรากฏ “บ่อยที่สุด”	GPT-2/GPT-4o, RoBERTa
WordPiece	เลือกคู่ที่เพิ่ม likelihood ของ training data มากที่สุด	BERT
Unigram LM (ULM)	เริ่มจาก vocab ใหญ่แล้วตัดหน่วยที่มีผลต่อ likelihood น้อยออก	SentencePiece, T5, ALBERT

ทั้งสามวิธีมีเป้าหมายเดียวกัน: หาสมดุลระหว่างขนาด *vocabulary* กับความยาวของลำดับ *token* — เพื่อให้โมเดล *generalize* ได้ดีและมีประสิทธิภาพเชิงคำนวณ

Thai Tokenization



Core Challenge

Thai text has NO spaces between words. Sentence: “ฉันทักคุณ” has no visible word boundary — the model must infer it from context and dictionary.



Dictionary-Based Word Segmentation

- Uses a pre-built lexicon of Thai words.
- Scans text and looks for valid dictionary entries.
- Fast and interpretable, but limited by vocabulary coverage.
- Out-of-vocabulary (OOV) words — new names, slang — are a key weakness.



Longest Matching Algorithm

- Greedy: at each position, match the longest possible dictionary word.
- Forward maximum matching (FMM): scan left-to-right.
- Backward maximum matching (BMM): scan right-to-left.
- Ambiguity problem: multiple valid segmentations may

```
PyThaiNLP: word_tokenize("ฉันทักคุณ", engine="newmm")
```